

Discrete Martingale Calculus + Exchangeability / Reverse Martingales / de Finetti

Work Handoff Design - Version 0.1

Implementation order is normative: Part I Discrete Martingale Calculus MUST be established before Part II Exchangeability / Reverse Martingales / de Finetti.

Primary mathematical coverage source: Achim Klenke, Probability Theory: A Comprehensive Course, 3rd ed. (2020).

Canonical Lean baseline: active StochLean pin. Current Mathlib-master observations in this document are reconnaissance only and never replace the pinned duplicate/API audit.

WORK HANDOFF SPECIFICATION - IMPLEMENTATION RULES ARE NORMATIVE

1. Purpose, Package Boundary, and Normative Status

Purpose. This document is the implementation contract for one combined functional package. It preserves the approved detailed design of the Discrete Martingale Calculus package and then appends the approved Exchangeability / Reverse Martingales / de Finetti design. The package is intentionally ordered: build and stabilize Part I first; only then build Part II.

- Klenke supplies coverage/provenance. Lean files are organized by mathematical function, never by chapter number.
- Part I owns only gaps in discrete martingale calculus that survive duplicate audit: named discrete stochastic integral if absent, predictable bracket / pathwise discrete quadratic variation if absent, finite-unbounded UI optional sampling if absent, Doob Lp and bracket-convergence gaps if absent, plus selected representation bridges.
- Part II owns exchangeability, symmetry, exchangeability sigma-fields, a thin reverse-time martingale adapter, conditional-iid vocabulary only if genuinely missing, and de Finetti. It MUST consume Part I martingale/convergence machinery rather than duplicate it.
- Mathlib-owned filtration, adaptedness, predictability, stopping times, conditional expectation/distribution, kernels, upcrossing, Doob decomposition, and existing convergence theorems are canonical dependencies and are not mirrored locally.
- Status vocabulary: REUSE, BRIDGE, BUILD, APPLICATION, DEFER, AUDIT-FIRST, EXTERNAL-CHECK. An AUDIT-FIRST item may become BUILD only after the hard duplicate gate is passed.

2. Global Non-Negotiable Design Rules

- Coverage rule: Klenke 3rd edition remains the completeness/provenance baseline for covered material. Do not silently omit a labeled definition/theorem/corollary or a load-bearing exercise. If not implemented, the ledger must record why.
- Functional organization rule: Never create Klenke/ChapterXX.lean or duplicate cross-chapter implementations. Every mathematical object has one canonical owner/module.
- No-duplicate rule: Before any public declaration is written, search in this exact order: (1) pinned Mathlib revision; (2) the entire local StochLean tree, including completed and concurrent functional packages; (3) the approved-external ledger. Equivalent usable semantics forbids a parallel implementation.
- Concurrent-package rule: If a canonical declaration is assigned to another package but not yet implemented, treat it as an explicit blocker or dependency. Do NOT create a temporary duplicate to make the current branch compile.
- No-error rule: Compilation is not mathematical validation. Do not weaken hypotheses, silently strengthen conclusions, substitute a surrogate model, encode the target conclusion as a field/hypothesis, or close goals through vacuity/uninhabited assumptions.
- No-semantic-ambiguity rule: Explicitly distinguish pointwise vs a.e.; bounded vs a.s.-finite stopping time; predictable bracket $\langle M \rangle$ vs pathwise quadratic variation $[M]$; ordinary vs reverse martingale interface; conditional-law equality a.e. vs raw pointwise equality; finite exchangeability vs infinite exchangeability.
- No-totalization-abuse rule: Public theorems must carry the true mathematical domain. Never exploit fallback values of conditional expectation, WithTop untop/default indices, division by zero, or other totalized operations as if they were mathematical values.
- Proof-integrity rule: no sorry, admit, project-defined axiom, conclusion-shaped assumption, or hidden trust boundary. Milestone theorems must pass axiom/dependency inspection.
- Canonical-representation rule: Use Mathlib-native Measure/ProbabilityMeasure, Kernel/condDistrib, Lp, Filtration, Adapted/StronglyAdapted, IsStronglyPredictable, IsStoppingTime, Martingale/Submartingale/Supermartingale, UniformIntegrable, iCondIndepFun, etc. Processes remain raw families $X : T \rightarrow \Omega \rightarrow E$.
- A.s.-version rule: Objects defined only up to null sets must have a.e./version-aware statements. Do not convert regular conditional distributions or directing measures into unjustified raw pointwise uniqueness.
- Generic-upstream rule: New non-model-specific mathematics should be placed in generic modules suitable for future Mathlib upstream. Do not lock generic results inside Galton-Watson, finance, Brownian, or de Finetti application namespaces.
- Minimal-import rule: production modules use narrow imports. Avoid blanket import Mathlib when a maintainable dependency closure exists.

- Source-correction rule: official errata and source-semantic corrections override OCR/printed shorthand. The implementation must encode the mathematically correct theorem.

3. Duplicate, Trust-Boundary, and Provenance Rules

HARD GATE: a new public declaration may be merged only after the duplicate/API audit is recorded. This gate applies again at every phase boundary because Mathlib, the local tree, and concurrently completed packages may have changed.

Source tier	Allowed use	Mandatory record
Pinned Mathlib	First-choice canonical implementation. REUSE whenever semantics/API are usable.	Exact module/declaration and active revision.
Local StochLean	Second search tier; includes Ch01-08, ProcessCore, earlier phases of this combined package, and concurrent approved packages.	Canonical owner, declaration path, whether reused or extended.
Approved external	May be reused/ported only after explicit mathematical and dependency audit.	Repo/version/declaration/trust status/license if relevant.
Unapproved external/current master	Reconnaissance only. Cannot justify a trust dependency or a REUSE status on the active pin.	Mark EXTERNAL-CHECK or AUDIT-FIRST.
Klenke	Coverage/source semantics, theorem assumptions, counterexamples, and acceptance targets.	Chapter/section/theorem or exercise provenance; note generalizations/corrections.

4. Combined Package Ownership and Mandatory Build Order

The combined package has two sequential parts. The second part is not an independent implementation stream.

Stage	Canonical owner	Hard dependency rule
Part I - Discrete Martingale Calculus	martingale transform/integral glue, bracket/QV, optional-sampling gaps, inequalities/convergence gaps	Must be built, audited, and exposed canonically before Part II reverse-martingale proofs are implemented.
Part II - Exchangeability / Reverse Martingales / de Finetti	exchangeability, symmetric sigma-fields, reverse-time adapter, conditional-iid interface, de Finetti	Must consume Part I ordinary martingale/upcrossing/UI/convergence engine. Reverse-specific copies are forbidden.
Foundations Ch01-08	UI/Vitali, condExp, condDistrib, Kernel, tail/independence, empirical CDF	Both parts must reuse. Missing foundation declarations are blockers, not invitations to duplicate.
Process Core	filtration, adapted/predictable, stopping/random-time semantics, process-law/path interfaces	Both parts must reuse. No local filtration/stopping/predictable mirrors.

Normative build order:

1. Run the full duplicate/API collision audit against pinned Mathlib -> entire local StochLean tree -> approved external.
2. Implement and close Part I mandatory gaps; run Part I semantic and CI gates.
3. Freeze canonical Part I public APIs needed by Part II, especially convergence/optional-sampling and any named discrete-integral/bracket objects.
4. Re-run duplicate/API audit before Part II, now including all newly completed Part I declarations.
5. Implement Part II only through canonical Part I/Foundation/ProcessCore APIs. Missing dependencies remain explicit blockers.
6. Run unified coverage, dependency, semantic, source, and minimal-import audits for the combined package.

PART I - DISCRETE MARTINGALE CALCULUS

Klenke primary provenance: Sections 9.2-9.4, Chapter 10, Chapter 11 (with branching applications treated as consumers).

5. Part I Functional Scope and Ownership

Function	Status	Canonical directive
Martingale/Sub/Super definitions	REUSE	Use Mathlib Probability.Martingale.Basic; no parallel predicates.
Discrete stochastic integral / transform object	AUDIT-FIRST -> BUILD-CANDIDATE	Create a named generic object only if active pin/local tree lacks a canonical public operation.
Klenke 9.39 transform stability	REUSE/BRIDGE	Wrap existing sum-transform theorems if a named integral object is added.
Binary martingale representation	AUDIT-FIRST/BUILD-CANDIDATE	Generic probability theorem only; CRR finance stays APPLICATION/DEFER.
Doob decomposition	REUSE	Use Mathlib Centering predictablePart/martingalePart.
Predictable quadratic variation <M>	AUDIT-FIRST -> BUILD if absent	Canonical bracket; never call it the same thing as pathwise [M].
Pathwise discrete quadratic variation [M]	AUDIT-FIRST -> BUILD if absent	Companion canonical object; connect to bracket by compensator theorem.
Covariation	BUILD-CANDIDATE	Future-proof optional addition; not a milestone blocker.
Bounded optional sampling/stopping	REUSE	Use Mathlib OptionalSampling / OptionalStopping.
UI + a.s.-finite unbounded stopping times	AUDIT-FIRST/BUILD-CANDIDATE	Separate theorem layer; bounded theorem may not be reused as if equivalent.
Upcrossing / weak Doob maximal	REUSE	Use Mathlib Upcrossing and OptionalStopping.
Doob Lp inequality, $p > 1$	AUDIT-FIRST/BUILD-CANDIDATE	If missing on active pin, implement upstream-quality generic theorem.
L1/UI martingale convergence	REUSE	Use Mathlib Convergence.
Bracket-based convergence criteria	BUILD/BRIDGE if missing	Klenke 11.13-11.14 layer; reuse canonical ordinary convergence engine.
Conditional Borel-Cantelli	REUSE/BRIDGE	Use existing martingale Borel-Cantelli if exact semantics match.
Galton-Watson martingale applications	APPLICATION	Consume existing Ch01-08 Galton-Watson; never duplicate branching code.

6. Discrete Stochastic Integral / Martingale Transform

Source semantics (Klenke Def. 9.37): for a predictable strategy H and process X , $(H \cdot X)_n = \sum_{m=1}^n H_m (X_m - X_{m-1})$. If a canonical named operation is absent after audit, StochLean may add one; otherwise this whole section is BRIDGE/REUSE.

- Preferred name if built: `discreteStochasticIntegral H X`. Do not use the unqualified name `stochasticIntegral` because the continuous Itô integral will be a different construction.
- Pure algebra first: integral at 0 is 0; one-step increment is $H_{n+1}(X_{n+1} - X_n)$; additivity/scalar laws; constant-one strategy gives $X_n - X_0$.
- Predictability indexing is essential: H_{n+1} must be $\mathcal{F}_n$ -measurable. Do not silently shift indices.
- Klenke “locally bounded” in this discrete theorem means sectionwise/each-time bounded, not localization by stopping times. If a predicate is really needed, use a name such as `IsSectionwiseBounded`; otherwise keep explicit hypotheses.
- For martingales, signed predictable transforms are allowed under the exact integrability/boundedness assumptions. For submartingales, nonnegativity of H is required; never drop it.

- The converse characterization via single-step predictable strategies is a BRIDGE target only if it produces reusable API; do not create conclusion-shaped assumptions.

7. Binary Martingale Representation

- Klenke § 9.4 distinguishes genuinely binary information increments from general martingales. Example 9.41 supplies a multi-valued counterexample to unrestricted representation.
- If no canonical theorem exists after audit, implement a generic binary-splitting representation theorem in Probability/Martingale/Representation/Binary.lean.
- CRR pricing/finance is APPLICATION/DEFER and must not leak financial structures into the generic theorem.
- The theorem hypotheses must encode the true binary information condition; “there exists a representation coefficient” cannot be used as an input hypothesis.

8. Doob Decomposition, $[M]$, and $\langle M \rangle$

Doob decomposition itself is REUSE through Mathlib predictablePart / martingalePart. New ownership is only the missing quadratic-variation interface.

- Klenke Def.10.3 “square variation process” is the predictable bracket $\langle M \rangle$, not the pathwise quadratic variation $[M]$. This terminology correction is normative.
- If absent, define predictableQuadraticVariation M F P (or bracket) so that $\langle M \rangle_0 = 0$ and $\Delta \langle M \rangle_{n+1} = E[(\Delta M_{n+1})^2 \mid \mathcal{F}_n]$.
- Prove predictability, monotonicity, and $M^2 - \langle M \rangle$ is a martingale. Prove the predictable uniqueness/compensator characterization.
- Independently, if absent, define pathwise discreteQuadraticVariation M by $[M]_n = \sum_{k \leq n} (\Delta M_{k+1})^2$; prove $[M]_0 = 0$ and $\Delta [M]_{n+1} = (\Delta M_{n+1})^2$.
- Under square-integrability, $[M] - \langle M \rangle$ is a martingale. This is the canonical bridge: $\langle M \rangle$ is the predictable compensator of $[M]$.
- Klenke Thm.10.4: derive $E[\langle M \rangle_n] = E[(M_n - M_0)^2]$, then variance corollaries under the exact mean hypotheses.
- Optional covariation $[M, N]$ and $\langle M, N \rangle$ may be added if cheap and clearly future-useful; otherwise defer to continuous martingale calculus.

9. Optional Sampling / Stopping Layers

- Bounded stopping-time optional sampling/stopping is REUSE via Mathlib OptionalSampling / OptionalStopping.
- Hard semantic distinction: bounded means a deterministic bound; a.s.-finite means $P(\tau < \infty) = 1$ and may be unbounded.
- Klenke Thm.10.21: for a UI martingale and a.s.-finite $\sigma \leq \tau$, $X_\sigma = E[X_\tau \mid \mathcal{F}_\sigma]$. If absent on the active pin, this is a BUILD-CANDIDATE.
- Implementation route: truncate $\tau_n = \tau \wedge n$, use bounded optional sampling, then pass to the limit using the canonical Ch01-08 UI/Vitali framework. Do NOT create a martingale-specific duplicate of Vitali/UI machinery.
- Klenke Lemma 10.20: the stopped family $\{X_\tau : \tau \text{ finite stopping time}\}$ of a UI martingale is uniformly integrable; add only if absent and reusable.
- Never evaluate X_τ on $\tau = \text{top}$ by relying on WithTop untop/default fallback. A.s.-finiteness must be used to remove the exceptional set mathematically.

10. Inequalities and Convergence

- Upcrossing and its crossing-time/gambling strategy machinery are REUSE from Mathlib Upcrossing.
- Weak Doob maximal inequality is REUSE when available on the active pin.
- Doob L_p inequality for $p > 1$ is AUDIT-FIRST. If missing, implement the generic sharp standard constant $(p/(p-1))^p$; do not add a Klenke-only special case.
- L_1 -bounded a.e. convergence and UI L_1 convergence are REUSE from Mathlib Convergence. Do not create a parallel MartingaleConvergenceTheorem module.
- Bracket layer: Klenke Lemma 11.13 $\langle M \rangle^\tau = \langle M \rangle^\tau$ and Thm.11.14 bracket-finite $\Rightarrow$ a.s. convergence are BUILD/BRIDGE only if absent. They must reuse the canonical ordinary martingale convergence engine.

- Conditional Borel-Cantelli is REUSE/BRIDGE from the existing martingale probability API; no second conditional-independence/Borel-Cantelli framework.

11. Part I Mandatory Semantic Regression Tests

1. Predictable indexing: H_{n+1} must be F_n -measurable; a strategy using the next increment itself must fail predictability.
2. Discrete integral recursion and $H \equiv 1$ identity.
3. Signed transform preserves martingales under assumptions but does not generically preserve submartingales without $H \geq 0$.
4. Sectionwise boundedness must not be confused with local-martingale/localization semantics.
5. Exhibit a martingale for which $[M]$ and $\langle M \rangle$ are genuinely different; simple symmetric random walk may be a sanity test but is insufficient as the only test.
6. Verify $M^2 - \langle M \rangle$ is martingale and each bracket increment is F_n -measurable.
7. Verify $\langle M^\tau \rangle = \langle M \rangle^\tau$ for a nontrivial bounded stopping time when bracket is implemented.
8. An a.s.-finite but unbounded stopping time must not typecheck/use the bounded theorem as if it had a deterministic bound; the UI extension should handle it if implemented.
9. Keep a counterexample preventing an unrestricted optional-sampling theorem for arbitrary a.s.-finite stopping times of non-UI martingales.
10. Doob L_p API must require $p > 1$ and retain the correct constant.
11. Positive-probability $\tau = \text{top}$ cases must not be masked by totalized untopt/default evaluation.
12. Klenke Example 9.41 must fail the binary-representation hypotheses.
13. Galton-Watson applications must import/consume the Ch01-08 canonical branching object only.

12. Part I Functional Module Ownership and Work Order

Candidate module	Create only if	Mandatory role
Probability/Martingale/DiscreteIntegral.lean	named canonical object survives duplicate audit	algebra + predictable transform bridges
Probability/Martingale/Representation/Binary.lean	generic binary theorem absent	binary representation; no finance
Probability/Martingale/QuadraticVariation/Discrete.lean	pathwise QV absent	$[M]$
Probability/Martingale/QuadraticVariation/Predictable.lean	bracket absent	$\langle M \rangle$, compensator, $M^2 - \langle M \rangle$
Probability/Martingale/QuadraticVariation/Covariation.lean	optional future-proof gap	non-blocking
Probability/Martingale/OptionalSampling/UniformIntegrable.lean	a.s.-finite UI extension absent	unbounded finite stopping-time layer
Probability/Martingale/Inequalities/DoobLp.lean	L_p inequality absent	generic $p > 1$ inequality
Probability/Martingale/Convergence/QuadraticVariation.lean	bracket criteria absent	stopped bracket + convergence criteria

Part I implementation order is normative:

1. Full duplicate audit, including Ch01-08 and ProcessCore canonical APIs.
2. Discrete integral named object/bridges only if needed.
3. Binary representation if needed.
4. Pathwise $[M]$ and predictable $\langle M \rangle$; establish canonical terminology and API.
5. Doob L_p only if absent.
6. UI finite-unbounded optional sampling only if absent, consuming Ch01-08 UI/Vitali.
7. Bracket stopping/convergence layer.
8. Part I coverage/semantic/dependency/minimal-import gate. Freeze the public API used by Part II.

PART II - EXCHANGEABILITY, REVERSE MARTINGALES & DE FINETTI

Klenke primary provenance: Chapter 12; future Chapter 13.4 is a second proof route and must reuse the same public de Finetti API.

13. Part II Ownership and Dependency Boundary

Reverse martingale is an interface/reindexing layer over the ordinary martingale engine established in Part I. It is not a second calculus.

Object/result	Owner
ordinary martingale/upcrossing/UL/convergence/optional calculus	Part I / Mathlib
filtration, stopping, predictability, process information-flow	ProcessCore / Mathlib
conditional expectation/distribution, Kernel, conditional independence	Ch01-08 / Mathlib
tail sigma-field / Kolmogorov zero-one	Ch01-08 / Mathlib
exchangeability, symmetry, exchangeability sigma-fields	Part II
reverse-time adapter and reverse convergence interface	Part II, implemented through Part I engine
empirical probability measure if absent	Part II canonical owner; real empirical CDF remains Ch01-08 view
conditional-iid vocabulary if missing	Part II thin wrapper over condDistrib + iCondIndepFun
de Finetti structural + directing measure theorems	Part II
full weak convergence of empirical measures	future Measure-Convergence package

14. Exchangeability - Canonical f.d.d.-First Semantics

- Klenke Def.12.1 uses finite permutations; Remark 12.2 gives the equivalent finite-dimensional formulation. Use f.d.d.-first semantics in v0.1 to avoid circular dependence on future product/path-space construction.
- IsExchangeable is AUDIT-FIRST -> BUILD likely. Prefer finite-permutation / pairwise-distinct finite-index tuple invariance using the existing finite-dimensional-law API.
- Do not create a custom finite-permutation structure if Equiv.Perm / Fin n permutations suffice.
- Prove iid => exchangeable; exchangeable => identically distributed; subfamily/reindex and common measurable coordinate transform preservation.
- Hard distinctions: exchangeable does not imply independent; identical one-time marginals do not imply exchangeable; finite exchangeability does not imply infinite extendibility.

15. Symmetry, Symmetrization, and Empirical Probability Measure

- Klenke Def.12.4 / Ex.12.5: implement symmetry as predicates, not a heavyweight structure. If absent, provide IsSymmetric / IsNSymmetric and a finite symmetrization operator A_n.
- Symmetrization API: finite-permutation invariance, idempotence, linearity/integrability bridges, and the exact form used by Klenke Thm.12.10 and 12.17.
- Klenke empirical probability measure $X_{i,n} = (1/n) \sum \delta_{X_i}$ is a more general object than the Ch01-08 empirical CDF. If no canonical empirical measure exists, Part II owns one generic implementation.
- Never define n=0 by division totalization. Use nonempty finite samples / Fin(n+1) or another domain that guarantees positive sample size.
- Bridge to Ch01-08: for real state, $X_{i,n}((-\infty, x])$ must recover the existing empirical CDF; no second empirical-CDF definition.
- Load-bearing Exercise 12.1.1 (symmetric function factors through empirical measure): distinguish plain set/function factorization from measurable factorization. Reuse existing Doob-Dynkin/factorization infrastructure where applicable.

16. Exchangeability Sigma-Fields and Tail Relation

- Build the decreasing exchangeability sigma-field family E_n and $E = \bigcap E_n$ from Klenke Def.12.6 if no equivalent local API exists.
- Establish $E_{n+1} \subseteq E_n$ and the decreasing-filtration orientation required by reverse martingales.
- Klenke Remark 12.8: connect E_n with empirical measure plus future coordinates when the empirical-measure layer exists.
- Klenke Remark 12.9 gives $T \subseteq E$ in general. Klenke Cor.12.18 gives only modulo-P/completion equality under exchangeability. Raw sigma-field equality $E=T$ is forbidden.
- Tail sigma-field is canonical foundation infrastructure; Part II imports it and never defines a second tail sigma-field.

17. Symmetrized Conditional Expectation and Reverse Martingale Adapter

- Klenke Thm.12.10: $E[\phi(X)|E_n]$ equals the finite symmetrization $A_n(\phi)$ under exchangeability. This is BUILD and must use the canonical conditional-expectation API.
- Klenke Def.12.11: backwards/reverse martingale is ordinary martingale with reversed order. Prefer OrderDual/reindexing or a very thin reducible wrapper; do not create NegativeNat/BackwardsTime machinery unless forced by a real gap.
- A public ReverseMartingale predicate, if introduced, must reduce to ordinary Martingale over a reversed/decreasing filtration. It must not own independent integrability/upcrossing/optional-stopping methods.
- Klenke Remark 12.12 automatic UI is REUSE/BRIDGE from the canonical conditional-expectation UI theorem.
- Klenke Thm.12.14 reverse martingale convergence: Part II owns the reverse theorem interface, but the proof engine MUST be Part I/Mathlib ordinary martingale convergence. Reprove no reverse upcrossing or reverse UI stack.
- Limit is $E[X_0 | \bigcap G_n]$; orientation $G_{n+1} \subseteq G_n$ is mandatory.

18. Exchangeable Means, Symmetrized LLN, and Hewitt-Savage

- Klenke Ex.12.13: exchangeable averages form a reverse martingale. BUILD as a key bridge.
- Ex.12.15: $n^{-1} \sum X_i \rightarrow E[X_1|E] = E[X_1|T]$ a.s. and in L1. BUILD/BRIDGE using reverse convergence.
- Ex.12.16 iid SLLN is APPLICATION/REUSE; never reprove the general SLLN here.
- Thm.12.17 symmetrized LLN for integrable finite-coordinate ϕ is BUILD and load-bearing for de Finetti.
- Cor.12.18 exchangeability-vs-tail mod-P theorem is BUILD; preserve modulo-null semantics.
- Cor.12.19 Hewitt-Savage is AUDIT-FIRST $\rightarrow$ BUILD likely; it must reuse canonical Kolmogorov/tail zero-one infrastructure.

19. Conditional Independence, Conditional IID, and de Finetti

- Klenke Def.12.20 conditional independence is REUSE: use Mathlib iCondIndepSets / iCondIndep / iCondIndepSet / iCondIndepFun and their two-object variants. No parallel predicate.
- If conditional identical-distribution / conditionally-iid vocabulary is missing, add only a thin wrapper: conditional independence + equality of conditional laws via condDistrib kernels a.e.
- Do not define conditional law by raw pointwise $P(X \in B | A)(\omega)$ and then demand equality for every ω . All version-sensitive equalities are a.e. with respect to the conditioning law/measure.
- Klenke Thm.12.24 structural de Finetti is BUILD: infinite exchangeability iff conditionally iid given a suitable sigma-field; positive direction may use exchangeability/tail sigma-fields.
- Klenke Thm.12.26 directing random probability measure is BUILD and MUST reuse condDistrib. Prefer Mathlib ProbabilityMeasure E as the value type if suitable; do not invent RandomProbabilityMeasure/DirectingMeasure structures without a real need.
- Directing-measure uniqueness is a.e./mixing-law uniqueness, not raw function pointwise equality.
- Klenke assumes Polish state in the directing-measure theorem. A Lean strengthening to StandardBorelSpace is allowed only when fully proved and explicitly documented as a source-generalization.
- Klenke Remark 12.27 test-integral convergence may be built as a bridge; full weak convergence of X_i belongs to the future Measure-Convergence package and must not be reimplemented here.

20. Part II Complete Functional Ledger

Source / function	Status	Directive
Def.12.1 exchangeability	AUDIT-FIRST -> BUILD likely	f.d.d.-first finite-permutation semantics
Rmk.12.2 f.d.d. criterion	BUILD/BRIDGE	equivalence with finite-permutation formulation
Def.12.4 symmetry	AUDIT-FIRST/BUILD	predicates only
Ex.12.5 symmetrization	BUILD	canonical finite symmetrization operator
Def.12.6 exchangeability sigma-fields	BUILD	decreasing family + intersection
Rmk.12.8 empirical-measure generation	BUILD/BRIDGE	depends on empirical measure
Rmk.12.9 $T \leq E$	BUILD/BRIDGE	reuse tail sigma-field
Thm.12.10 symmetrized CE	BUILD	reuse condExp
Ex.12.1.1 symmetric factorization	BUILD/BRIDGE	load-bearing; separate measurable factorization
Def.12.11 reverse martingale	REUSE/BRIDGE	order-dual/reindex ordinary Martingale
Rmk.12.12 automatic UI	REUSE/BRIDGE	consume Ch01-08 CE/UI theorem
Ex.12.13 means are reverse martingale	BUILD	key bridge
Thm.12.14 reverse convergence	REUSE/BRIDGE; minimal BUILD only if needed	must consume Part I engine
Ex.12.15 exchangeable LLN	BUILD/BRIDGE	limit by E/T
Thm.12.17 symmetrized LLN	BUILD	load-bearing
Cor.12.18 E vs T mod P	BUILD	never raw equality
Cor.12.19 Hewitt-Savage	AUDIT-FIRST -> BUILD likely	reuse tail zero-one
Def.12.20 conditional independence	REUSE	Mathlib conditional independence
conditional identical law / conditional iid vocabulary	AUDIT-FIRST/BRIDGE	condDistrib a.e. + iCondIndepFun
Thm.12.24 structural de Finetti	BUILD	exchangeable iff conditionally iid
Def.12.25 empirical random probability measure	AUDIT-FIRST/BUILD-CANDIDATE	generic empirical measure; no n=0 totalization
Thm.12.26 directing-measure de Finetti	BUILD	reuse condDistrib
Rmk.12.27 test-integral convergence	BUILD/BRIDGE	full weak convergence deferred
Ch.13.4 alternate de Finetti proof	FUTURE REUSE	same public theorem/API; no second de Finetti

21. Part II Mandatory Semantic Regression Tests

1. iid implies exchangeable.
2. $X_n = Z$ shows exchangeable does not imply independent.
3. $X_1 = Z, X_2 = Z, X_3 = -Z$ (nondegenerate symmetric Z) shows equal marginals do not imply exchangeable.
4. Finite without-replacement two-point law is exchangeable but cannot be infinitely extended; finite exchangeability must not instantiate infinite de Finetti.
5. Finite permutation implementation must genuinely represent the generated finite-permutation action; adjacent swaps are enough only if generation is formally proved.
6. $T \leq E$ in general; raw $E = T$ is forbidden. Exchangeable equality is only modulo P/completion.
7. Decreasing orientation $G_{\{n+1\}} \leq G_n$ is verified.
8. Reverse martingale convergence depends on Part I/Mathlib ordinary engine; dependency inspection must not reveal copied reverse upcrossing/UI/convergence code.
9. Reverse limit equals conditional expectation on intersection G_n .
10. Conditional independence is not monotone in the conditioning sigma-field; retain the intermediate $\sigma(X+Y)$ counterexample pattern.
11. Conditionally iid implies exchangeable.
12. Directing kernel/random measure equality is a.e./version-aware, never unjustified pointwise equality.
13. Empirical probability measure cannot be built from a zero-size sample by division fallback.
14. For $E = R$, empirical-measure CDF agrees with the Ch01-08 empirical CDF.

15. Bernoulli de Finetti acceptance: a directing Y in $[0,1]$ gives conditional finite laws $\text{Ber}(Y)^{\otimes J}$.
16. Future Ch.13.4 proof reuses the same public de Finetti theorem.
17. Any Polish $\rightarrow$ StandardBorel strengthening must carry a source-generalization note and pass kernel/measurability proof checks.

22. Unified Functional Module Layout

Candidate module	Owner / condition
Probability/Martingale/DiscreteIntegral.lean	Part I; only if named object gap survives audit
Probability/Martingale/Representation/Binary.lean	Part I; only if generic theorem absent
Probability/Martingale/QuadraticVariation/Discrete.lean	Part I; only if [M] absent
Probability/Martingale/QuadraticVariation/Predictable.lean	Part I; only if $\langle M \rangle$ absent
Probability/Martingale/OptionalSampling/UniformIntegrable.lean	Part I; only if a.s.-finite UI theorem absent
Probability/Martingale/Inequalities/DoobLp.lean	Part I; only if $p > 1$ theorem absent
Probability/Martingale/Convergence/QuadraticVariation.lean	Part I; bracket convergence gaps
Probability/Exchangeability/Basic.lean	Part II; exchangeability core if absent
Probability/Exchangeability/Symmetry.lean	Part II; symmetry/symmetrization
Probability/Exchangeability/SigmaAlgebra.lean	Part II; E_n/E /tail bridges
Probability/Martingale/Reverse.lean	Part II; deliberately thin adapter only
Probability/EmpiricalMeasure/Basic.lean	Part II; only if generic empirical measure gap
Probability/Exchangeability/ConditionalIID.lean	Part II; only thin vocabulary gap
Probability/Exchangeability/HewittSavage.lean	Part II; only if theorem gap
Probability/Exchangeability/DeFinetti.lean	Part II; structural + directing-measure theorem

Forbidden local mirrors:

- Mathlib Martingale/Basic, Centering, OptionalSampling, OptionalStopping, Upcrossing, Convergence; Process Filtration/Adapted/Predictable/Stopping; conditional expectation/distribution/kernel; conditional independence; existing tail sigma-field / zero-one infrastructure.

23. Work Execution Protocol and Unified Acceptance Gate

- Coverage audit: every ledger row and load-bearing exercise is resolved against pinned Mathlib, the complete current local tree, and approved external sources.
- API audit: each BUILD has one canonical owner, explicit natural domain, a.e./version semantics, and no collision with another functional package.
- Part I implementation: implement only confirmed gaps in the normative Part I order; close semantic tests and freeze public APIs needed by Part II.
- Part II preflight: rerun duplicate audit including completed Part I. Unfinished Part I dependencies are blockers; no temporary duplicates.
- Part II implementation: exchangeability/reverse/de Finetti only through canonical foundation/ProcessCore/Part I APIs.
- Semantic audit: run all mandatory regression tests in Sections 11 and 21. Counterexamples are part of the contract, not optional demonstrations.
- CI gate: lake build; project warnings treated as errors where practical; no sorry/admit/project axiom; lint; inspect axioms/dependency closure of milestone theorems.
- Minimal-import gate: replace umbrella imports with the narrow stable dependency closure before milestone completion.
- Coverage gate: every BUILD is implemented; every AUDIT-FIRST is resolved; EXTERNAL-CHECK is either approved or explicitly deferred with no hidden dependency; source mapping is complete.
- Upstream/maintenance gate: if a local bridge/result is later upstreamed to Mathlib or an approved canonical source, remove the local duplicate and migrate callers rather than maintaining two versions.

24. Milestone Completion Definition

The combined package is complete only when ALL of the following hold:

- Part I Discrete Martingale Calculus is complete before Part II is declared complete; Part II reverse-martingale theorems use the canonical Part I ordinary martingale engine.
- No duplicate canonical definitions/theorems exist across Mathlib, local StochLean, or approved external dependencies.
- Bracket $\langle M \rangle$ and pathwise $[M]$ are either canonically available and reused or correctly implemented with distinct semantics and regression tests.
- Bounded vs a.s.-finite stopping-time theorem layers are not conflated; no WithTop fallback is used as mathematics.
- Exchangeability is f.d.d.-correct, not marginal-only; finite vs infinite exchangeability is guarded by counterexamples.
- Exchangeability sigma-field vs tail sigma-field is represented with correct modulo-null semantics.
- Conditional independence is reused from Mathlib; conditional-iid and directing-measure statements are a.e./version-correct.
- Structural and directing-measure de Finetti theorems have one public API that future alternative proofs reuse.
- All source assumptions/generalizations/corrections are recorded; no OCR/textbook shorthand silently changes the formal statement.
- Final build is clean, trusted dependency/axiom boundary is documented, and all mandatory semantic tests compile.

25. Reference Map for Work

Mathematical source	Primary coverage in this package
Klenke 9.2-9.4	martingale transform / discrete stochastic integral / binary representation
Klenke 10.1	Doob decomposition; predictable bracket semantics
Klenke 10.2-10.3	bounded vs UI finite stopping-time optional sampling
Klenke 11.1-11.2	upcrossing, Doob maximal/Lp, martingale convergence, bracket criteria
Klenke 11.3	branching applications only
Klenke Chapter 12	exchangeability, reverse martingales, exchangeability sigma-fields, conditional iid, de Finetti
Klenke 13.4	future alternate proof of same de Finetti API; no duplicate theorem
Mathlib canonical modules to audit	Probability.Martingale.Basic/Centering/OptionalSampling/OptionalStopping/Upcrossing/Convergence; Probability.Process.Predictable/Stopping; Probability.Independence.Conditional; Probability.Kernel.CondDistrib; current local Ch01-08 and ProcessCore